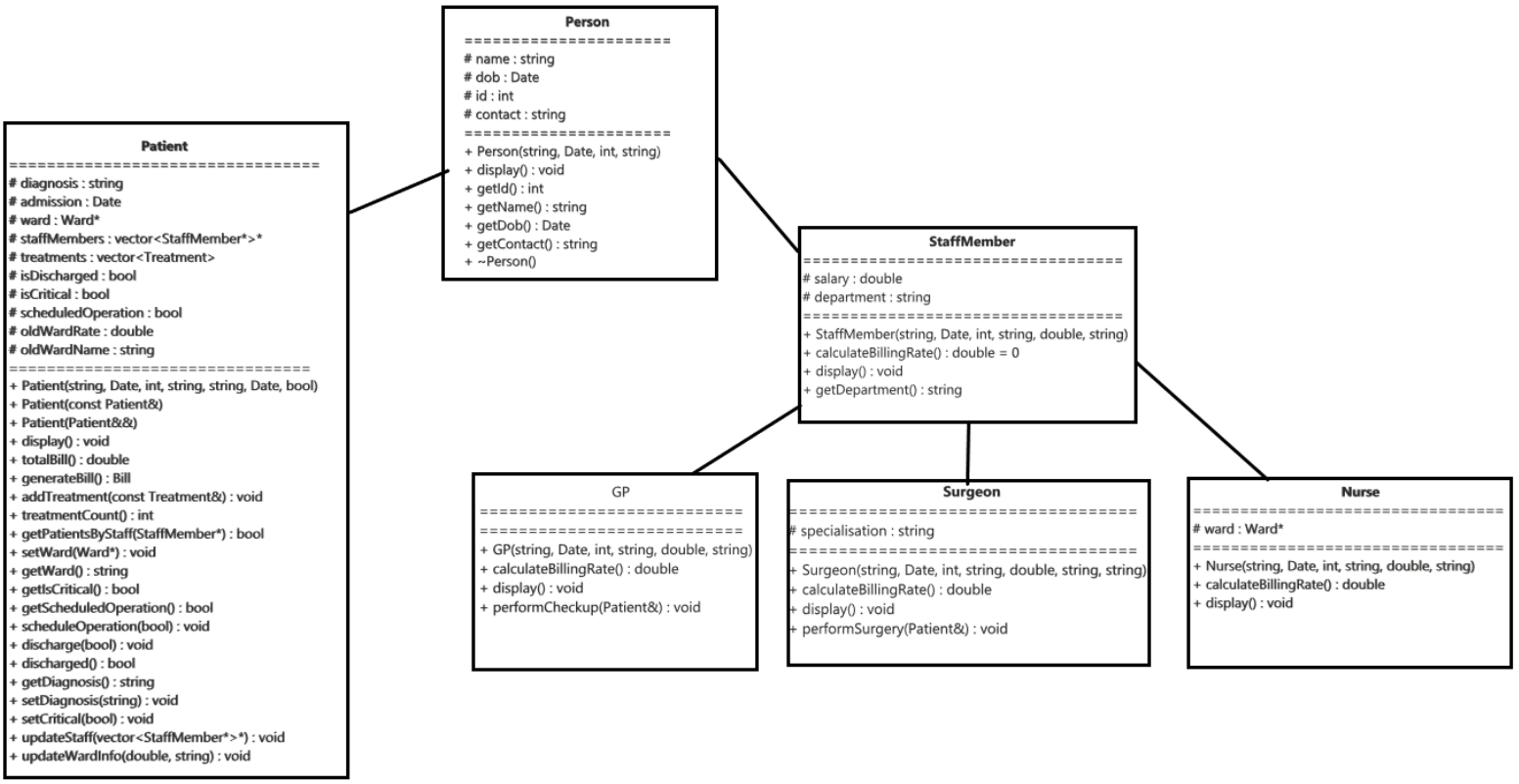
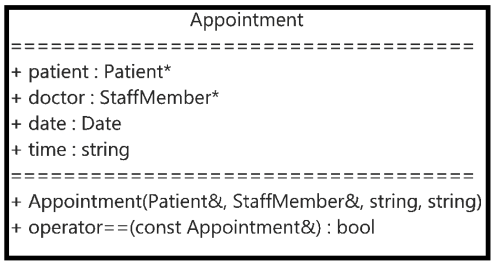
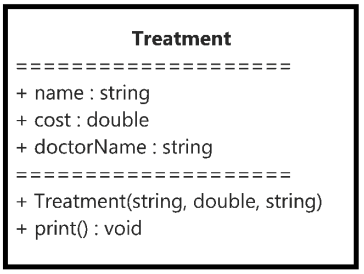
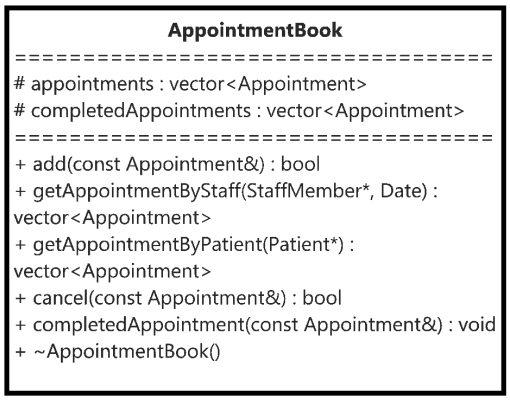
MediCore Design

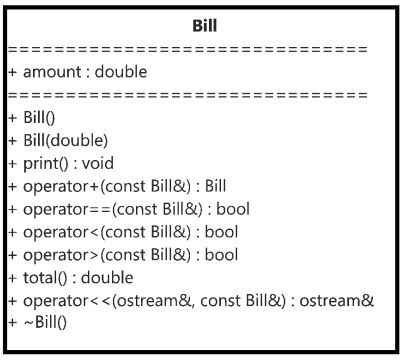
Class diagram and relationships:



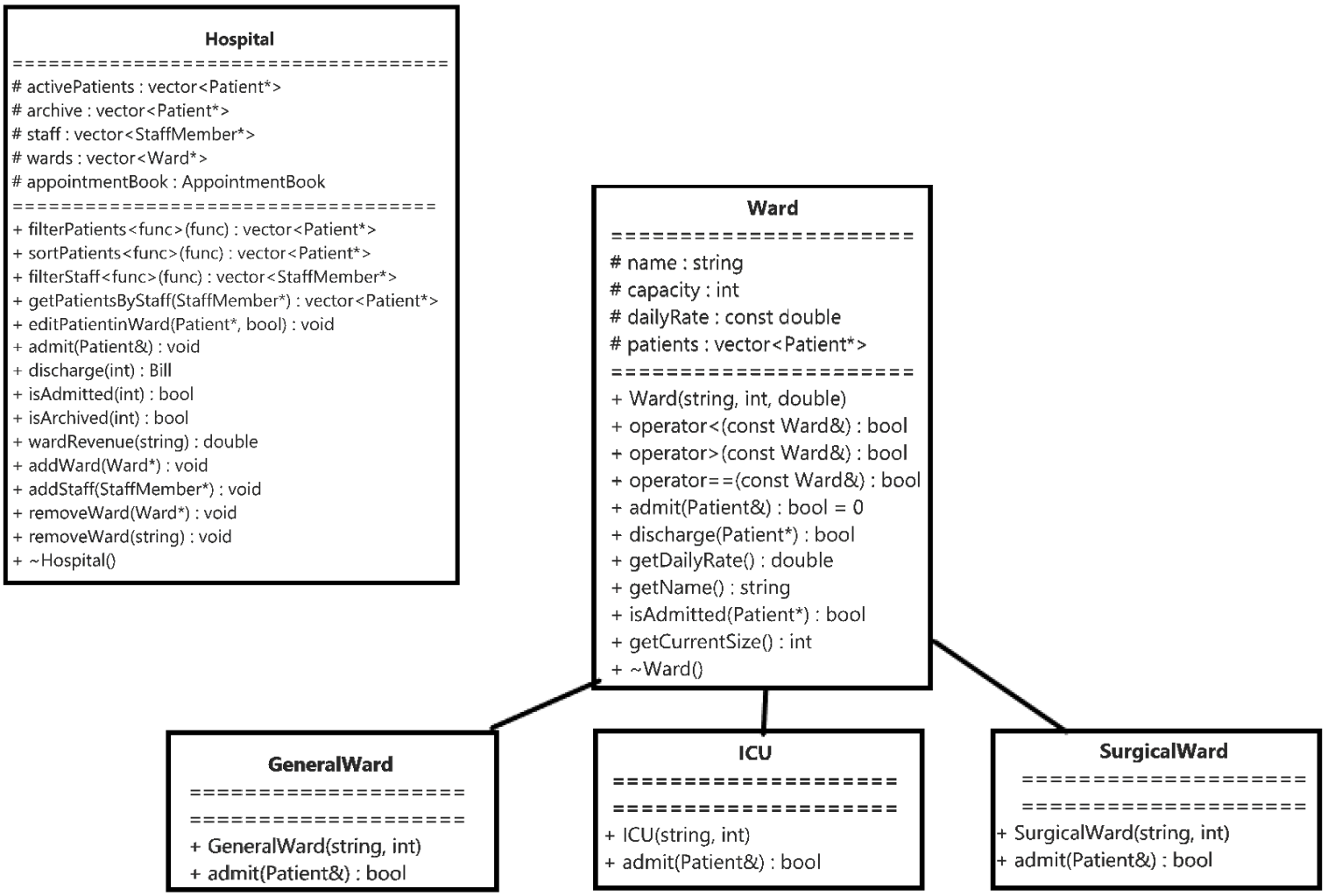








By: Hamza Faisal Ahmed 32406



==========

<u>Class hierarchy:</u>

All classes defined in separate files, except person and ward, which contain the definition of their child classes too.

Person has 2 child classes: patient and staffMember. Staff member has further 3 classes: GP nurse and surgeon.

Ward and StaffMember are abstract classes, and the function display is overridden by all classes. Similarly, calculateBillingRate is also overridden by all of staffmember's child classes.

Ward has 3 child classes: ICU, general ward and surgical ward. All these classes override the admit function, to check if their admit criteria is met, otherwise they reject admitting that patient.

<u>Design choices:</u>

By: Hamza Faisal Ahmed 32406

Polymorphism used for person and ward classes to avoid repeated code and to allow access through parent class pointers.

Abstract classes used to ensure no incomplete or irrelevant data exists.

Templates used to allow generic lambda functions to ensure flexible sort and filtering. Operators overloaded to allow natural comparisons among wards and also bills.

Code split into headers and implementation files to allow better visibility and help during debugging.

Overview:

The hospital class manages patients, staff, wards, billing and appointments. All classes are interconnected by this one controller class.
The person class carries all the data for the people in the hospital. The staff members and their details, along with their rates, as well as the patients, who the class manages to not only store, but also edit their records.

Wards have 3 types, and each ward has a separate admission criteria. Wards are owned by the hospital class, and each have their own billing rate too. They also have overloaded operators to compare their occupancy percentage.

AppointmentBook manages scheduling and makes sure that all appointments are valid, making sure there are no double bookings. It also tracks completed appointments for the records.

Billing ensures the bills can be combined and stored in their own type. Bills are generated by discharge or directly through patient class itself.

The hospital class also keeps archived patients, to calculate total ward revenues, and can also filter records however you like using lambda functions.

Some other design decisions include a custom Date class, which can take a string parameter and parse it, but the string must be exactly formatted (yyyy-mm-dd), otherwise it will throw an exception, which will also be the case if the date entered is not valid (eg 40th january).

A current date global variable exists to let the classes calculate the number of days between today and admission dates.

Each patient stores their old ward info when discharged, so that they can show up in the ward revenue call. While they are active, they store a pointer to the ward they are assigned to, enabling them to get ward rates independently of the hospital class. This was done to ensure that the ward data is available for archived patients, as when discharged their ward pointer will be set to null.

By: Hamza Faisal Ahmed 32406

Patients also store a reference to the staffMember list in hospital, to allow them to get their current rate while calculating the bill. This is because each treatment can have separate doctors and the treatment itself stores only the name.

Copy and move constructors are also implemented for patients, but since pointers are stored, they are not demonstrated explicitly in main.

Fees are fixed for all doctors and wards (hardcoded)

Hospital manages memory by deleting all wards, as its the only thing it owns. The patients and staff were shown to be created separately in main, which is why I let their ownership stay within the main function, where after main completes all test cases, they are deleted.

Overall, most of the objects like Appointment for example stores pointers to each other rather than copies, to ensure clean and fast access.

By: Hamza Faisal Ahmed 32406